

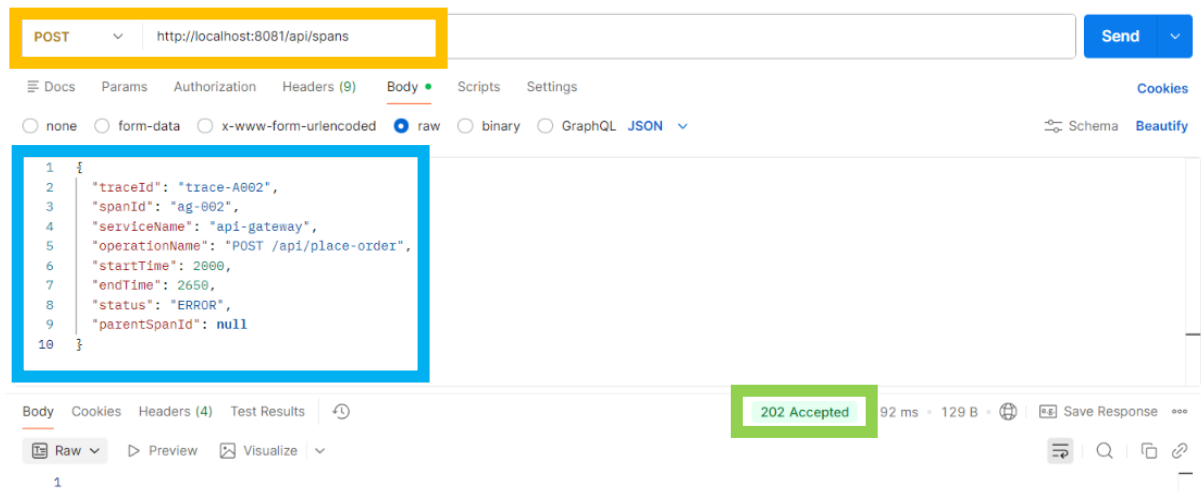
System in Action!

For demonstration purposes we are considering 3 simulated microservices: api-gateway, order-service, and payment-service.

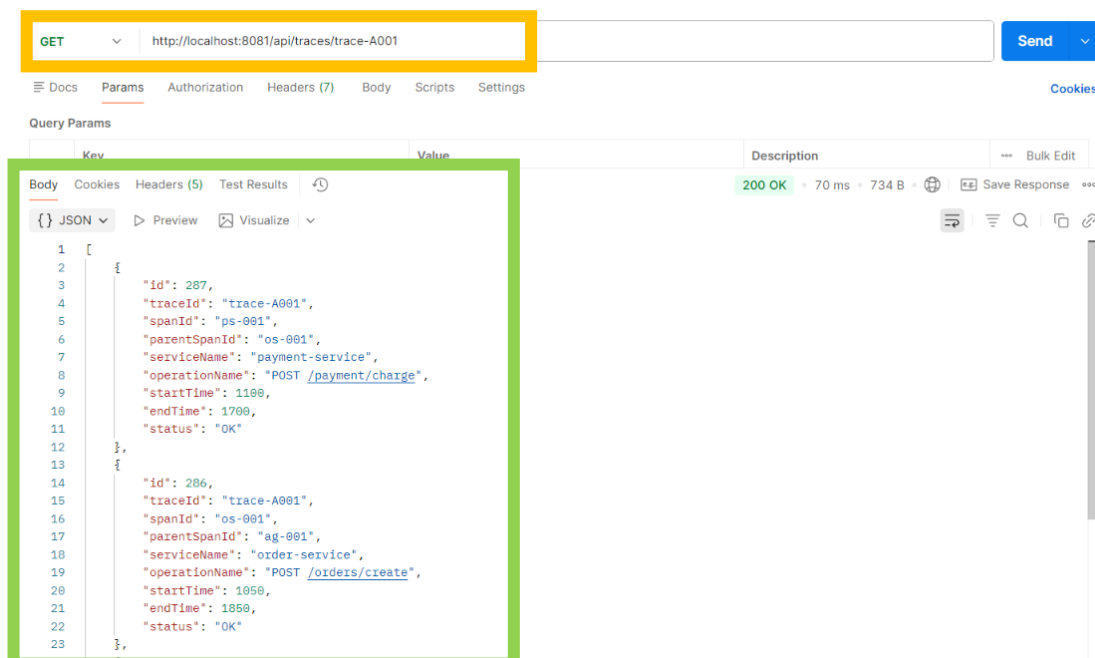
The spans reported by these microservices are summarized in the below table:

Trace ID	Span ID	Parent Span ID	Service	Operation	Start Time	End Time	Duration	Status
trace-A001	ag-001	null	api-gateway	POST /api/place-order	1000	1900	900	OK
trace-A001	os-001	ag-001	order-service	POST /orders/create	1050	1850	800	OK
trace-A001	ps-001	os-001	payment-service	POST /payment/charge	1100	1700	600	OK
trace-A002	ag-002	null	api-gateway	POST /api/place-order	2000	2650	650	ERROR
trace-A002	os-002	ag-002	order-service	POST /orders/create	2050	2600	550	ERROR
trace-A002	ps-002	os-002	payment-service	POST /payment/charge	2100	2500	400	ERROR

1. Data flowing in: Microservices reports its execution — trace ID, span ID, parent span ID, service name, operation, duration, and status sent to the collector by making a http post request to the exposed API endpoint.



2. Trace lookup: Querying by a trace ID returns every span recorded under that trace ID — including the operations executed by each service, and how long it took — giving a complete record of everything that happened during that request.



Full Response:

```
[
  {
```

```
"id": 287,

"traceId": "trace-A001",

"spanId": "ps-001",

"parentSpanId": "os-001",

"serviceName": "payment-service",

"operationName": "POST /payment/charge",

"startTime": 1100,

"endTime": 1700,

"status": "OK"

},

{

  "id": 286,

  "traceId": "trace-A001",

  "spanId": "os-001",

  "parentSpanId": "ag-001",

  "serviceName": "order-service",

  "operationName": "POST /orders/create",

  "startTime": 1050,

  "endTime": 1850,

  "status": "OK"

},

{

  "id": 285,

  "traceId": "trace-A001",

  "spanId": "ag-001",

  "parentSpanId": null,

  "serviceName": "api-gateway",

  "operationName": "POST /api/place-order",

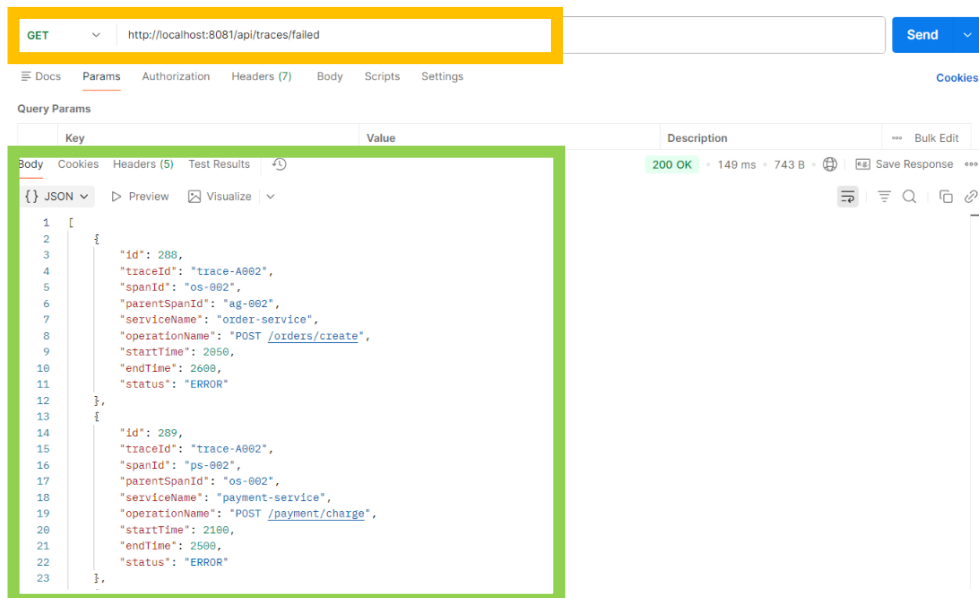
  "startTime": 1000,

  "endTime": 1900,

  "status": "OK"

}
```

3. Surface the failure: A single call surfaces every failed span across all services — no log diving, no manual correlation.



Full Response:

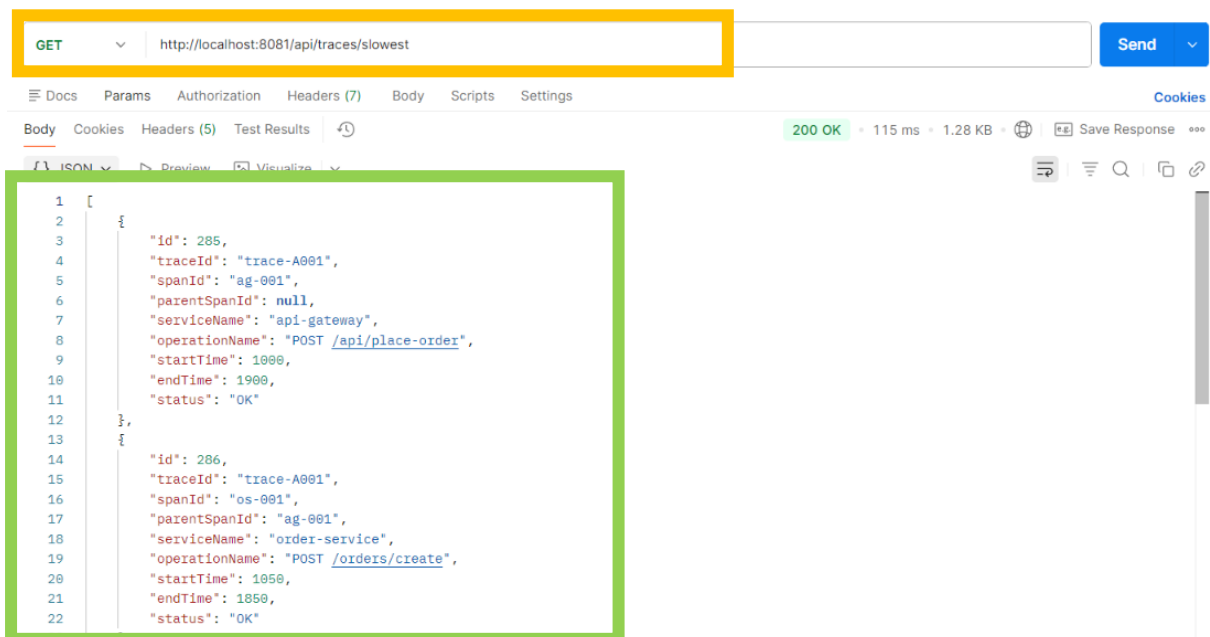
```
[
  {
    "id": 288,
    "traceId": "trace-A002",
    "spanId": "os-002",
    "parentSpanId": "ag-002",
    "serviceName": "order-service",
    "operationName": "POST /orders/create",
    "startTime": 2050,
    "endTime": 2600,
    "status": "ERROR"
  },
  {
    "id": 289,
    "traceId": "trace-A002",
    "spanId": "ps-002",
    "parentSpanId": "os-002",
    "serviceName": "payment-service",
    "operationName": "POST /payment/charge",
```

```

    "startTime": 2100,
    "endTime": 2500,
    "status": "ERROR"
  },
  {
    "id": 290,
    "traceId": "trace-A002",
    "spanId": "ag-002",
    "parentSpanId": null,
    "serviceName": "api-gateway",
    "operationName": "POST /api/place-order",
    "startTime": 2000,
    "endTime": 2650,
    "status": "ERROR"
  }
]

```

4. Find the bottleneck: Slowest operations ranked automatically — immediately shows where the system is bleeding time.



GET http://localhost:8081/api/traces/slowest

200 OK • 115 ms • 1.28 KB

```

1 [
2   {
3     "id": 285,
4     "traceId": "trace-A001",
5     "spanId": "ag-001",
6     "parentSpanId": null,
7     "serviceName": "api-gateway",
8     "operationName": "POST /api/place-order",
9     "startTime": 1000,
10    "endTime": 1900,
11    "status": "OK"
12  },
13  {
14    "id": 286,
15    "traceId": "trace-A001",
16    "spanId": "os-001",
17    "parentSpanId": "ag-001",
18    "serviceName": "order-service",
19    "operationName": "POST /orders/create",
20    "startTime": 1050,
21    "endTime": 1850,
22    "status": "OK"
  }
]

```

Full Response:

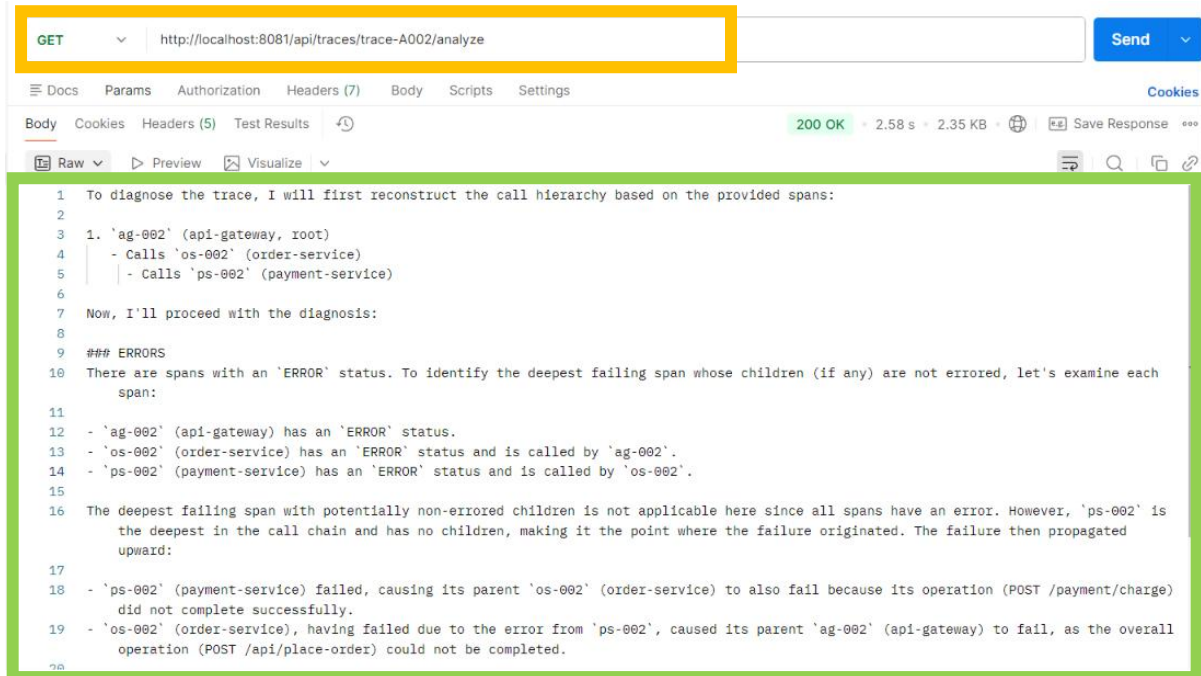
```
[
```

```
{
  "id": 285,
  "traceId": "trace-A001",
  "spanId": "ag-001",
  "parentSpanId": null,
  "serviceName": "api-gateway",
  "operationName": "POST /api/place-order",
  "startTime": 1000,
  "endTime": 1900,
  "status": "OK"
},
{
  "id": 286,
  "traceId": "trace-A001",
  "spanId": "os-001",
  "parentSpanId": "ag-001",
  "serviceName": "order-service",
  "operationName": "POST /orders/create",
  "startTime": 1050,
  "endTime": 1850,
  "status": "OK"
},
{
  "id": 290,
  "traceId": "trace-A002",
  "spanId": "ag-002",
  "parentSpanId": null,
  "serviceName": "api-gateway",
  "operationName": "POST /api/place-order",
  "startTime": 2000,
  "endTime": 2650,
  "status": "ERROR"
},
```

```
{
  "id": 287,
  "traceId": "trace-A001",
  "spanId": "ps-001",
  "parentSpanId": "os-001",
  "serviceName": "payment-service",
  "operationName": "POST /payment/charge",
  "startTime": 1100,
  "endTime": 1700,
  "status": "OK"
},
{
  "id": 288,
  "traceId": "trace-A002",
  "spanId": "os-002",
  "parentSpanId": "ag-002",
  "serviceName": "order-service",
  "operationName": "POST /orders/create",
  "startTime": 2050,
  "endTime": 2600,
  "status": "ERROR"
},
{
  "id": 289,
  "traceId": "trace-A002",
  "spanId": "ps-002",
  "parentSpanId": "os-002",
  "serviceName": "payment-service",
  "operationName": "POST /payment/charge",
  "startTime": 2100,
  "endTime": 2500,
  "status": "ERROR"
}]
```

5. The AI diagnosis: The same trace ID, now analysed by an LLM — root cause, affected service, and recommendations returned in plain English.

(A) Analysing trace-A002



Full response:

To diagnose the trace, I will first reconstruct the call hierarchy based on the provided spans:

1. `ag-002` (api-gateway, root)
 - Calls `os-002` (order-service)
 - Calls `ps-002` (payment-service)

Now, I'll proceed with the diagnosis:

ERRORS

There are spans with an `ERROR` status. To identify the deepest failing span whose children (if any) are not errored, let's examine each span:

- `ag-002` (api-gateway) has an `ERROR` status.
- `os-002` (order-service) has an `ERROR` status and is called by `ag-002`.
- `ps-002` (payment-service) has an `ERROR` status and is called by `os-002`.

The deepest failing span with potentially non-errored children is not applicable here since all spans have an error. However, `ps-002` is the deepest in the call chain and has no children, making it the point where the failure originated. The failure then propagated upward:

- `ps-002` (payment-service) failed, causing its parent `os-002` (order-service) to also fail because its operation (POST /payment/charge) did not complete successfully.

- `os-002` (order-service), having failed due to the error from `ps-002`, caused its parent `ag-002` (api-gateway) to fail, as the overall operation (POST /api/place-order) could not be completed.

LATENCY

After identifying the error propagation, let's look at the latency:

- Total trace duration is essentially the duration of `ag-002`, which is 650ms.
- `os-002` took 550ms, which is significant but less than the total trace duration.
- `ps-002` took 400ms, which is substantial but given its parent's duration, it does not stand out as unusually high on its own.

Given the error condition, the durations are somewhat overshadowed by the fact that all spans failed. However, `ps-002` and `os-002` have significant durations, but without more context on what is "usual," we cannot definitively call them bottlenecks based solely on the provided data.

HEALTHY

Since there are errors present in the trace, it did not complete normally.

In summary, the root cause of the failure seems to originate from `ps-002` (payment-service), which then caused its parent spans to fail. There are no clear latency bottlenecks without more context, but durations of the erroring spans are notable.

(B) Analysing trace-A001

GET http://localhost:8081/api/traces/trace-A001/analyze

200 OK • 2.44 s • 2.52 KB

Body Cookies Headers (5) Test Results

Raw Preview Visualize

```
1 To reconstruct the call hierarchy based on the 'parentSpanId', we start from the root (the span with 'parentSpanId=null') and work our way
2 down:
3 1. 'ag-001' (api-gateway, POST /api/place-order) is the root span because its 'parentSpanId' is 'null'.
4 2. 'os-001' (order-service, POST /orders/create) is called by 'ag-001' because 'os-001's 'parentSpanId' is 'ag-001'.
5 3. 'ps-001' (payment-service, POST /payment/charge) is called by 'os-001' because 'ps-001's 'parentSpanId' is 'os-001'.
6
7 The call hierarchy from top to bottom is:
8 - 'ag-001' (api-gateway)
9   - 'os-001' (order-service)
10    - 'ps-001' (payment-service)
11
12 Now, let's diagnose this trace:
13
14 1. **ERRORS**: There are no spans with a status of 'ERROR'. All spans have a status of 'OK'.
15
16 2. **LATENCY**: To identify any latency issues, let's compare the durations:
17   - 'ag-001' (api-gateway) has a duration of 900ms.
18   - 'os-001' (order-service) has a duration of 800ms.
19   - 'ps-001' (payment-service) has a duration of 600ms.
20
21 Given that 'ag-001' is the root and its duration is the sum of the durations of its child ('os-001') and the child's child ('ps-001')
22 plus any additional processing time, we should consider if the durations of 'os-001' and 'ps-001' are reasonable components of
23 'ag-001's total duration.
24
25 The total duration of 'os-001' and 'ps-001' is 1400ms (800ms + 600ms), which exceeds the duration of 'ag-001' (900ms). This suggests
26 that either the durations provided do not account for all time spent in the services or there's an overlap in how time is being
```

Full Response

To reconstruct the call hierarchy based on the `parentSpanId`, we start from the root (the span with `parentSpanId=null`) and work our way down:

1. `ag-001` (api-gateway, POST /api/place-order) is the root span because its `parentSpanId` is `null`.
2. `os-001` (order-service, POST /orders/create) is called by `ag-001` because `os-001`'s `parentSpanId` is `ag-001`.
3. `ps-001` (payment-service, POST /payment/charge) is called by `os-001` because `ps-001`'s `parentSpanId` is `os-001`.

The call hierarchy from top to bottom is:

- `ag-001` (api-gateway)
- `os-001` (order-service)
- `ps-001` (payment-service)

Now, let's diagnose this trace:

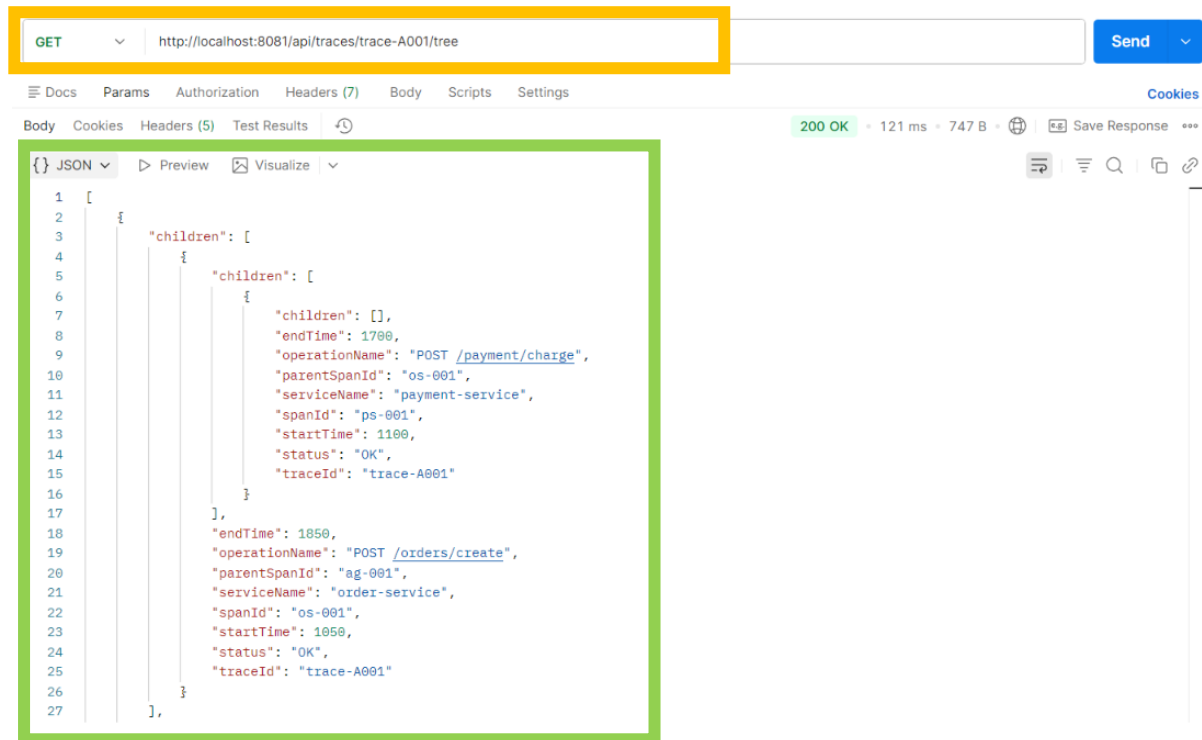
1. ****ERRORS****: There are no spans with a status of `ERROR`. All spans have a status of `OK`.
2. ****LATENCY****: To identify any latency issues, let's compare the durations:
 - `ag-001` (api-gateway) has a duration of 900ms.
 - `os-001` (order-service) has a duration of 800ms.
 - `ps-001` (payment-service) has a duration of 600ms.

Given that `ag-001` is the root and its duration is the sum of the durations of its child (`os-001`) and the child's child (`ps-001`) plus any additional processing time, we should consider if the durations of `os-001` and `ps-001` are reasonable components of `ag-001`'s total duration.

The total duration of `os-001` and `ps-001` is 1400ms (800ms + 600ms), which exceeds the duration of `ag-001` (900ms). This suggests that either the durations provided do not account for all time spent in the services or there's an overlap in how time is being measured or reported. However, without more context, we can't definitively say there's a latency issue based solely on the provided data. The durations provided for the child spans exceed the parent's duration, which might indicate an issue in how durations are measured or reported rather than an inherent latency problem in the services themselves.

3. ****HEALTHY****: Given there are no errors and the question about latency is more about the consistency of the reported durations rather than an obvious bottleneck, we can conclude that the trace completed normally based on the information provided. The reported durations of the child spans exceeding the parent span's duration is unusual but does not necessarily indicate a performance issue within the context of this trace. It could suggest a need to review how durations are measured or reported across services.

6. Trace tree: Given a trace ID, this endpoint reconstructs the full call hierarchy for that request — linking each span to the one that called it — showing exactly how the request flowed across services, and making it easy to see where in that chain a failure originated.



Full Response:

```
[
  {
    "children": [
      {
        "children": [
          {
            "children": [],
            "endTime": 1700,
            "operationName": "POST /payment/charge",
            "parentSpanId": "os-001",
            "serviceName": "payment-service",
            "spanId": "ps-001",
            "startTime": 1100,
            "status": "OK",
            "traceId": "trace-A001"
          }
        ],
        "endTime": 1850,
        "operationName": "POST /orders/create",
        "parentSpanId": "ag-001",
        "serviceName": "order-service",
        "spanId": "os-001",
        "startTime": 1050,
        "status": "OK",
        "traceId": "trace-A001"
      }
    ]
  }
]
```

```
    }  
  ],  
  "endTime": 1850,  
  "operationName": "POST /orders/create",  
  "parentSpanId": "ag-001",  
  "serviceName": "order-service",  
  "spanId": "os-001",  
  "startTime": 1050,  
  "status": "OK",  
  "traceId": "trace-A001"  
}  
],  
"endTime": 1900,  
"operationName": "POST /api/place-order",  
"parentSpanId": null,  
"serviceName": "api-gateway",  
"spanId": "ag-001",  
"startTime": 1000,  
"status": "OK",  
"traceId": "trace-A001"  
}  
]
```

Flow in services: api-gateway -> order-service -> payment-service